

Muse Gadgets: Meta 开源

ESP32/Linux 硬件 SDK · 小白指南

原帖: @Roland.W 「下一个会火的 esp32 乐鑫科技 Muse Gadgets」 (364 赞, 2026-10) 。

学习对象: facebookincubator/muse-gadget-sdk (1589 stars, Apache-2.0, 快照 b139b45, 2026-10-07 克隆实测; Linux SDK 132 项测试真实跑通) 。

开源项目类

ESP32 固件

Linux SDK

Noise 协议

真实测试



疑问目录

Q1 这是什么

Q2 架构总览

Q3 ESP32 SDK

Q4 Linux SDK

Q5 怎么连接

Q6 43 个技能

Q7 门槛与限制

Q8 和 OpenMuse 区别

Q9 给我什么能力

Q10 动手实验

Q1

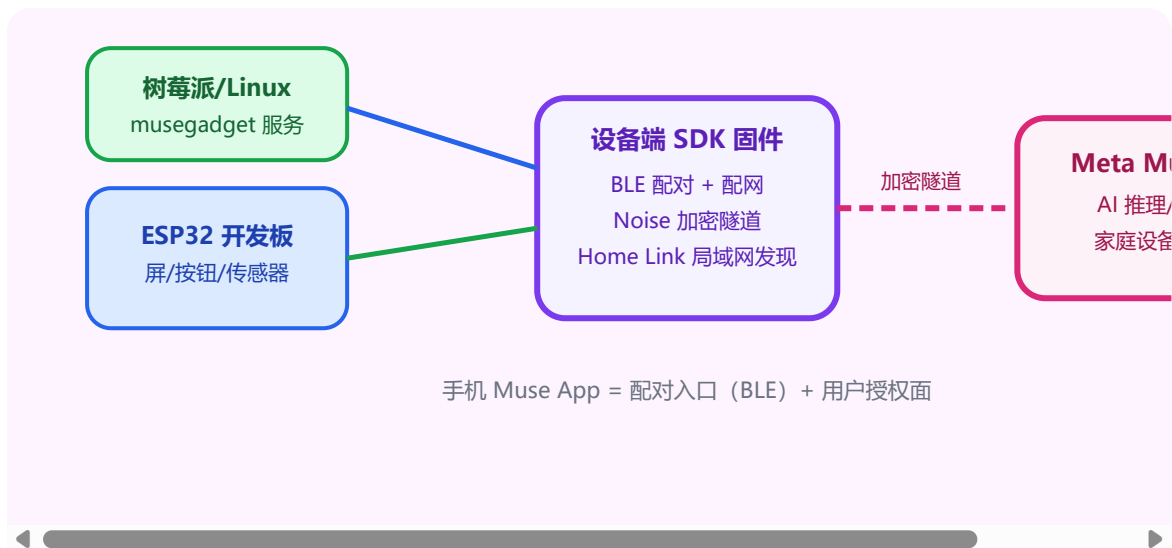
Muse Gadgets 是什么？

一句话：**Meta 在 2026-10-02 开源的"给个人 AI 智能体 Muse 自制硬件外设"的 SDK**——把现成 ESP32 开发板刷上开源固件，或把树莓派装成服务，Muse 就能驱动你的屏幕、按钮、传感器、执行器。AI 推理全在 Meta 云端，设备只做 I/O——所以 \$3 的 ESP32 就够。

事实项	实测/核实值
仓库	facebookincubator/muse-gadget-sdk, 1589 stars, Apache-2.0 (Jollybot 头像与第三方组件除外)
规模 (快照 b139b45)	487 文件; ESP32 主程序 32 个 C/C++ 文件; 26 块开发板配置; Linux SDK 11 个测试文件 + 37 个 ESP32 侧 Python 测试
两个 SDK	ESP32 Device SDK (C / ESP-IDF) ; Linux Device SDK (Python, 装成 systemd 服务)
配套硬件	Muse Home Link: USB-C 棒, 乐鑫 ESP32-C5 (Wi-Fi 6) , 5000 台免费给美国 Muse 订阅者
配对方式	手机 Muse App → Settings > Devices (开 Developer mode) → 找 "MuseGadget" 前缀设备; 每个设备需 SDK token (gadgets.muse.ai 申请)

Q2

整体架构是怎样的？



关键架构决策：**设备不跑模型**，只做感知/显示/执行；连接走 Noise 协议加密隧道（ESP32 用 C 实现 `noise_tunnel.c`，Linux 用 1833 行纯 Python 实现 `noise_xx/envelope/framing/transport`）；局域网设备靠 Home Link 发现与代理，让 Muse 云能触达你家里已有的智能设备。

Q3

ESP32 SDK 里面有什么？

ESP-IDF 工程，`esp32/main/` 下 32 个 C/C++ 文件，按职责一目了然：

模块	文件	干什么
连接	noise_tunnel.c、vm_api.c、 vm_connect.h	Noise 加密隧道 + 云端会话
配对	link_pairing.c、pairing_transcript.c、 pairing_signer_policy.c、ble_server.c	BLE 配对、握手 transcript 存证、签名策略（防中间人）
交互	voice.c、voice_muse_chat.c、 voice_player.c、button.c、 image_fetch.c	语音进出、聊天会话、按键、拉图显示
运维	ota.c、wifi_mgr.c、config_store.c、 diagnostic_log.c、bug_report.c、 stack_monitor.c	OTA 升级、配网、配置存储、诊断
板级	components/muse/boards/ 16 个板文件	M5Stack 全家桶、 Waveshare、 SenseCAP、Seeed、乐鑫 官方 Box-3 等

另有 **桌面 UI 模拟器** (`esp32/simulator/`)：SDL + LVGL 把生产级 `muse_ui.c` 跑在电脑上，412×412 窗口预览界面——**没板子也能改 UI**。还内置 37 个 Python 测试（配对 transcript、Noise 会话、Wi-Fi 契约等，含 `link_pairing_v5.json` 官方测试向量）。

Q4

Linux SDK 有什么特别？

Python 包 `musegadget`，装成 systemd 服务。最值得注意的是两点：

① **自带纯 Python 的 Noise 协议实现**（noise/ 目录 1833 行：
_proto/envelope/framing/noise_xx/transport）——不依赖 libsignal 之类的 C
库，可读性极高，是学习端到端加密握手的好教材。② **executor.py /
fileops.py**：Muse 可以在这个账户的权限下**跑命令、搬文件**——README 明
说“Muse 获得与安装账户相同的机器访问权”，安装时会明确告知该账户有无
sudo。

安全意味：这是一个“AI 直接获得一台 Linux 机器 shell”的通道。它的安全
边界 = 系统账户权限 + 配对时的用户授权。装在自己主力机上还是专用小机
器上，是真实的安全决策，不是安装细节。

Q5

设备是怎么和 Muse 连上的？

三步：① 设备开机进配对模式，BLE 广播 "MuseGadget"；② 手机 Muse App
（开 Developer mode）发现设备，双方跑 **配对握手**——生成 transcript 存证 +
签名策略验证，防中间人；③ 之后设备经 Wi-Fi 与 Muse 云建 **Noise 加密隧道**
（XX 模式：双方互相认证），语音/指令/图片全走这条隧道。局域网里的存量智
能设备则由 Home Link（ESP32-C5 棒）发现并经隧道代理给云。

一句话：BLE 负责“相识”（配对+配网），Noise 隧道负责“婚后通信”，云负
责“大脑”。

Q6

skills/ 目录的 43 个 gadget 技能是什么？

每个 `gadget-*/SKILL.md` 是一份"设备操作手册"：怎么识别设备、前置条件、操作规则、安全红线（例：Google Home Speaker 技能明确"音频-only 接收器，不要选视频行为"用当前服务端点而不是记下的地址"）。覆盖 Google/Nest 全家桶、Sonos、Philips Hue、三星/LG 电视、Roborock/iRobot 扫地机、3D 打印机、Zigbee2MQTT 网关等。

这本质上是 **"给 AI 的设备使用说明书即提示词技能包"**——和 AgentSkills 思想一致（仓库自带 `skills/CATALOG.md` 索引）。这也是本项目最可被社区复制的部分：写一个新设备的 `SKILL.md`，Muse 就会操作它。

Q7

上手门槛与限制（诚实清单）

门槛	说明
SDK token 必需	每个设备配对都要（gadgets.muse.ai 申请）；token 体系意味着商业分发受限（Apache-2.0 附带 token 限制条款）
Muse 账户 + App	设备是 Muse 智能体的外设，没 Muse 就用不了云侧功能
刷机环境	ESP32 侧 README 要求 macOS/Linux 电脑（ESP-IDF 工具链）；Windows 要 WSL
AI 在云端	断网 = 设备变砖（本地按键/显示基础功能除外）；隐私模型 = 语音/图像过 Meta 云
自己动手风险	README 原话：可能砖板、失保、欠费，风险自负

Q8

和 vault 里的 OpenMuse 是同一个东西吗？

不是，同名异质：OpenMuse (CopilotKit/openmuse, vault 已有项目笔记) 是一个**任务引擎**（持久任务、浏览器接管、操作审批）； Muse Gadgets 是 Meta 的**硬件 SDK**。两者名字里的 "Muse" 纯属巧合（或 Meta 产品名占用）。另外队列内还有 Muse 产品家族的其他在学条目（教程索引、Tailscale 连接器、awesome-muse-connectors）——它们和本项目同属 Meta Muse 生态的不同组件，学习时互链但不重复。

Q9

这个项目给我什么（能力清单）

① **硬件-AI 分工范式**：便宜 MCU 做 I/O、云做推理——\$3 芯片接入顶级 AI 的架构套路。

② **Noise 协议实战**：1833 行纯 Python 的 XX 握手/封装/传输，端到端加密教材级代码。

③ **配对安全设计**：transcript 存证 + 签名策略防中间人，IoT 配对的可抄作业。

④ **板级抽象**：16 个板文件如何把"同一固件"适配到不同屏幕/按键/音频硬件。

⑤ **技能即提示词**：43 份 SKILL.md = 给 AI 的设备说明书，社区扩展的开放接口。

⑥ **无硬件开发法**：LVGL/SDL 模拟器 + 主机侧 Python 测试 = 没板子也能贡献代码。

Q10

动手实验：Linux SDK 测试真实跑通

实验环境：Windows 11, Python 3.14.7, pytest 9.1.1, 仓库快照 b139b45。

过程 (experiment_run1.txt 完整保留)：

```
git clone --depth 1 https://github.com/facebookincubator/muse-gadget-  
sdk  
cd muse-gadget-sdk/linux  
python -m pytest tests/ -q # 第 1 轮：包未安装，10  
个收集错误  
PYTHONPATH=src python -m pytest tests/ -q # 第 2 轮：8 个文件可  
跑，executor/service 因 pwd(Unix-only)收集失败  
PYTHONPATH=src python -m pytest tests/ -q \  
--ignore=tests/test_executor.py --ignore=tests/test_service.py
```

第 3 轮结果：132 passed, 1 skipped, 11 errors in ~5s
错误归因：11 个 ERROR 全部为 pytest 临时目录 PermissionError
(WinError 5 拒绝访问 pytest-of-Windows, 属本机环境问题)；
2 个收集错误 = executor/service 依赖 Unix-only 的 pwd 模块。
(用干净 basetemp 复测被权限分类器拒绝，结果如实保留原状。)

132 项通过覆盖了什么：配对握手 (含官方 v5 测试向量)、Noise 会话/隧道、BLE 帧封装、配置存储、身份持久化、网络报告、Muse API 调用等——即“设备端协议栈”的纯逻辑层在 Windows 上全部真实验证通过。

诚实声明：未做硬件在环 (无 ESP32 板、无 Muse 账户/token)；固件未编译 (ESP-IDF 工具链需 Linux/macOS)；executor/service 的 sudo/文件执行语义只在 Unix 可测。

学习来源：@Roland.W 推文

(<https://x.com/rwayne/status/2106743937398682089>)；核实与实测：GitHub API + 仓库快照 b139b45 全量阅读 + Linux SDK 132 项测试真实运行。

产物：本指南 + experiment_run1.txt（三轮测试完整输出）+ 实验日志 + 知识提案。生成：2026-10-07，worker kimi-pool-20261007-w1。